

IUT-FV - GI1
Projet de Programmation C
Projet 1 : Gestion des Produits
Premier semestre de l'année académique 2018-2019
Dr. KENMOGNE Edith Belise

Le but de ce projet est d'écrire en langage C un programme de gestion d'une liste de produits. Un produit est caractérisé par son code, son libellé (une chaîne de caractères) et le prix unitaire.

Partie I : Structures de Données

- 1) Définir un type *PRODUIT*. Illustrer cette définition par un schéma.
- 2) Définir un type *LISTE_PRODUIT* constitué de trois champs : Le premier champ est un tableau de produits. Le deuxième champ est la taille du tableau. Le troisième champ est le nombre de produits stockés dans le premier champ. Le troisième champ représente aussi la prochaine case libre du tableau. Il faudrait l'appeler *libre*. Illustrer la définition de *LISTE_PRODUIT* par un schéma.

Partie II : Opérations sur un produit

Définir les fonctions suivantes.

- 1) *PRODUIT* lire_produit1()* lit les caractéristiques d'un produit au clavier et retourne l'adresse d'une zone mémoire contenant les valeurs lues.
- 2) *void lire_produit2(PRODUIT *p)* lit les caractéristiques d'un produit au clavier et les stocke dans **p*.
- 3) *void affiche_prod1(PRODUIT *p)* affiche le produit **p* à l'écran.
- 4) *void affiche_prod2(PRODUIT pro)* affiche le produit *pro* à l'écran.

Partie III : Opérations sur une liste de produits

- 1) *LISTE_PRODUIT creation_lp(int n)* crée une liste de produits pouvant contenir *n* produits. Le premier champ de l'objet crée est une zone mémoire obtenue dynamiquement et pouvant contenir *n* produits.
- 2) *int vide_lp(LISTE_PRODUIT lp)* retourne TRUE si le tableau de la liste de produits *lp* est vide (ne contient pas de produit) et FALSE sinon.
- 3) *int plein_lp(LISTE_PRODUIT lp)* retourne TRUE si le tableau de la liste de produits *lp* est plein et FALSE sinon.
- 4) *int ajout_lp(LISTE_PRODUIT *plp, PRODUIT p)* copie le produit *p* dans la première case libre du premier champ de la liste de produits **plp* et retourne TRUE (pour dire que l'ajout a réussi). La fonction doit retourner FALSE si la liste de produits **plp* est pleine.
- 5) *int seach_lp1(LISTE_PRODUIT lp, char *code)* recherche le produit dont le code du produit associé est *code* et retourne la position dudit produit s'il existe dans *lp* et -1 sinon.
- 6) *PRODUIT* seach_lp2(LISTE_PRODUIT lp, char *code)* recherche le produit dont le code du produit associé est *code* et retourne un pointeur sur ledit produit s'il existe dans *lp* et NULL sinon.
- 7) *int delete_lp(LISTE_PRODUIT *plp, char *code)* recherche le produit dont le code est *code*, supprime ledit produit de **plp* et retourne TRUE (pour dire que la suppression a réussi) et retourne FALSE si le produit recherché n'existe pas dans **plp*.
- 8) *int trie_selection(LISTE_PRODUIT *plp)* trie par sélection le tableau de **plp* sur les prix.

Partie IV : programme principal et menu

Ecrire la fonction principale *main*. Il pourrait afficher le menu suivant dès son exécution. Si l'utilisateur entre un chiffre qui n'est pas compris entre 1 et 6, le menu est ré-affiché. Si l'utilisateur entre un chiffre compris entre 1 et 6, le traitement approprié est effectué suivi de l'affichage des résultats dudit traitement qui est à son tour suivi d'une invitation à retourner au menu ou à quitter l'application : Choisissez 0 pour revenir au menu et 6 pour quitter.....

MENU

1. Ajouter un produit
2. Supprimer un produit
3. Trier les produits par prix croissant
4. Afficher les produits
5. Rechercher un produit
6. Quitter

Choisissez un chiffre :.....

Partie V : Enrichissement du menu

Il s'agit de permettre l'enregistrement d'une liste de produits dans un fichier et le chargement du contenu d'un fichier de produits dans une liste de produits. Pour ce faire, vous devez définir les deux fonctions suivantes :

- *void to_liste_produit(LISTE_PRODUIT *plp, char *nom_fichier)* qui stocke tous les produits du fichier dans la liste de produits **plp*.
- *void to_file(char *nom_fichier, LISTE_PRODUIT lp)* qui crée un fichier et stocke tous les produits de la liste de produits *lp* dans le fichier créé.

MENU

1. Ajouter un produit
2. Supprimer un produit
3. Trier les produits par prix croissant
4. Afficher les produits
5. Rechercher un produit
6. Ouvrir le fichier de produits
7. Enregistrer dans un fichier
8. Quitter

Choisissez un chiffre :.....

Partie VI : Amélioration du menu et extension des fonctionnalités

IUT-FV - GI1
Projet de Programmation C
Projet 2 : Gestion des Etudiants
Premier semestre de l'année académique 2018-2019
Dr. KENMOGNE Edith Belise

Le but de ce projet est d'écrire en langage C un programme de gestion d'une liste d'étudiants. Nous considérons ici un fichier texte dont chaque ligne comprend : (1) le matricule d'un étudiant, (2) le nom de l'étudiant, et quatre notes. Cette structure est illustrée comme suit :

```
m0000 Messi 15 17 12 16
m0001 Tamo 17 18 11 15
m0002 Douala 16 15 15 14
m0003 Tang 9 14 5 17
```

Partie I : Structures de Données

- 1) Définir un type *ETUDIANT*. Illustrer cette définition par un schéma.
- 2) Définir un type *LISTE_ETUDIANT* constitué de trois champs : Le premier champ est un tableau d'étudiants. Le deuxième champ est la taille du tableau. Le troisième champ est le nombre d'étudiants stockés dans le premier champ. Le troisième champ représente aussi la prochaine case libre du tableau. Il faudrait l'appeler *libre*. Illustrer la définition de *LISTE_ETUDIANT* par un schéma.

Partie II : Opérations sur un étudiant

Définir les fonctions suivantes.

- 1) *ETUDIANT* lire_etudiant1()* lit les caractéristiques d'un étudiant au clavier et retourne l'adresse d'une zone mémoire contenant les valeurs lues.
- 2) *void lire_etudiant2(ETUDIANT *pe)* lit les caractéristiques d'un étudiant au clavier et les stocke dans **pe*.
- 3) *void affiche_etud1(ETUDIANT *pe)* affiche les champs de l'étudiant **pe* à l'écran.
- 4) *void affiche_etud2(ETUDIANT e)* affiche les champs de l'étudiant *e* à l'écran.
- 5) *float moyenne(ETUDIANT e)* retourne la moyenne de l'étudiant *e*.

Partie III : Opérations sur une liste d'étudiants

- 1) *LISTE_ETUDIANT creation_lp(int n)* crée une liste d'étudiants pouvant contenir *n* étudiants. Le premier champ de l'objet crée est une zone mémoire obtenue dynamiquement et pouvant contenir *n* étudiants.
- 2) *int vide_le(LISTE_ETUDIANT le)* retourne TRUE si le tableau de la liste *le* est vide (ne contient pas d'étudiant) et FALSE sinon.
- 3) *int plein_le(LISTE_ETUDIANT le)* retourne TRUE si le tableau de la liste d'étudiants *le* est plein et FALSE sinon.
- 4) *int ajout_le(LISTE_ETUDIANT *ple, ETUDIANT e)* copie l'étudiant *e* dans la première case libre du premier champ de la liste d'étudiants **ple* et retourne TRUE (pour dire que l'ajout a réussi). La fonction doit retourner FALSE si la liste d'étudiants **ple* est pleine.
- 5) *int seach_le1(LISTE_PRODUIIT le, char *code)* recherche l'étudiant dont le code est *code* et retourne la position dudit étudiant s'il existe dans *le* et -1 sinon.
- 6) *PRODUIIT* seach_lp2(LISTE_PRODUIIT le, char *code)* recherche l'étudiant dont le code est *code* et retourne un pointeur sur ledit étudiant s'il existe dans *lp* et NULL sinon.
- 7) *int delete_le(LISTE_ETUDIANT *ple, char *code)* recherche l'étudiant dont le code est *code*, supprime ledit étudiant de **ple* et retourne TRUE (pour dire que la suppression a réussi) et retourne FALSE si le produit recherché n'existe pas dans **ple*.
- 8) *int trie_selection(LISTE_ETUDIANT *ple)* trie par sélection le tableau de **ple* sur les moyennes.

Partie IV : programme principal et menu

Ecrire la fonction principale *main*. Il pourrait afficher le menu suivant dès son exécution. Si l'utilisateur entre un chiffre qui n'est pas compris entre 1 et 6, le menu est ré-affiché. Si l'utilisateur entre un chiffre compris entre 1 et 6, le traitement approprié est effectué suivi de l'affichage des résultats dudit traitement qui est à son tour est suivi d'une invitation à retourner au menu ou à quitter l'application : Choisissez 0 pour revenir au menu et 6 pour quitter.....

MENU

1. Ajouter un etudiant
2. Supprimer un etudiant
3. Trier les étudiants par moyenne croissante
4. Afficher les étudiants
5. Rechercher un étudiant
6. Quitter

Choisissez un chiffre :.....

V Enrichissement du menu

Il s'agit de permettre l'enregistrement d'une liste d'étudiants dans un fichier et le chargement du contenu d'un fichier d'étudiants dans une liste d'étudiants. Pour ce faire, vous devez définir les deux fonctions suivantes :

- *void to_liste_etudiant(LISTE_ETUDIANT *ple, char *nom_fichier)* qui stocke tous les étudiants du fichier dans la liste d'étudiants **ple*.
- *void to_file(char *nom_fichier, LISTE_ETUDIANT le)* qui crée un fichier et stocke tous les étudiants de la liste d'étudiants *le* dans le fichier crée.

MENU

1. Ajouter un etudiant
2. Supprimer un etudiant
3. Trier les étudiants par moyenne croissante
4. Afficher les étudiants
5. Rechercher un étudiant
6. Ouvrir le fichier des étudiants
7. Enregistrer dans un fichier
8. Quitter

Choisissez un chiffre :.....

Partie VI : Amélioration du menu et extension des fonctionnalités